

Using Gemini's Guided Learning Tool to Learn Lecture Notes

<https://gemini.google.com/share/47906148679d>

Executive Summary: I used Gemini's Guided Learning tool to develop a tool to help extend learning of lecture notes. I tested this tool on the notes from lectures 19 and 20 which covered transformer architectures and variants. Overall, I was very satisfied with the utility of the tool. It did a good job of quickly refreshing me on the concepts covered in the lecture, while challenging me later on to show my learning through increasingly difficult questions. I think the guided learning mode on Gemini is particularly helpful for this as I did my other special participation task using the normal version and I found it difficult to write the correct prompt to get the model to verify my learning properly. I also saw no hallucination - everything referenced in the summaries and questions was based on content covered in the lecture notes. I would highly recommend using this specific tool, or using the guided learning tool in Gemini in general to all students.

To use this tool, I would recommend opening up the link I pasted, uploading the relevant lecture notes and prompting the following: "run me through that entire process again except refer to these lecture notes. Disregard the previous lecture notes on the transformer I attached - we are now covering [write topic]".

I will show my entire conversation with comments below. I started by sending the lecture notes and instructing Gemini on how I wanted to learn:

Prompt: Your task is to tutor students in a deep learning class at UC Berkeley (CS182) on transformers. Please refer to the lecture notes I have attached, and do not add any information that is not covered in the lecture notes. Use whatever techniques you feel are best to allow students to gain a deep understanding of transformer architecture and use.

Response: This is a fantastic topic. Let's explore the Transformer architecture, its components, and its influential variants like GPT-3 and BERT, all based on the concepts covered in your CS182 lecture notes¹¹¹. I'll ask guiding questions along the way to ensure we build a deep understanding.

The Transformer architecture, introduced in the paper "Attention Is All You Need"², typically consists of an **encoder** and a **decoder**³³. Modern language models often simplify this, focusing on just the encoder (like BERT⁴) or just the decoder (like GPT-3⁵).

To begin our discussion, which of the following areas would you like to explore first?

1. 🧠 **Transformer Core Architecture & Flow:** We can start with the basic encoder-decoder structure, the role of components like Multi-Head Attention, Feed Forward networks, and the Add & Norm layers⁶⁶⁶⁶.
2. 🕒 **Positional Encoding (ROPE):** We can dive into how models use techniques like **ROPE** (Rotary Position Embedding) to ensure the model understands the order of tokens by making the attention score depend on the *relative position* of keys ($\vec{k}$) and queries ($\vec{q}$)⁷⁷⁷⁷.
3. 🆚 **GPT-3 vs. BERT:** We can compare and contrast these two dominant architectures, including their training objectives (e.g., masked attention vs. masked token prediction) and their use cases (generation vs. fine-tuning)⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸⁸.

Comments: Gemini demonstrates an initial understanding of the task at hand. In addition, it clearly summarizes the material in the lecture notes into 3 key sections without adding anything extraneous. Good start

Prompt: Cover all topics in one response and then test my understanding however you feel is best. We want to be sure that my understanding is deep enough for the standards of CS182

Response: This is a great goal for a CS182 student. The Transformer architecture is central to modern deep learning. We will cover the core architecture, the role of positional encoding like ROPE, and the differences between influential variants like GPT-3 and BERT, using only the information from your lecture notes.

1. The Core Transformer Architecture 🧠

The original Transformer architecture is built on an **Encoder-Decoder** structure¹.

A. Encoder and Decoder Blocks

Both the Encoder and Decoder are composed of identical layers repeated N times (e.g., 6 times)²²²².

4. **Encoder Layer Components:**
 1. **Multi-Head Attention**³³.
 2. **Feed Forward Network (FFN)**⁴.

5. Decoder Layer Components:

1. **Masked Multi-Head Attention** (Self-Attention)⁵.
2. **Multi-Head Attention** (Encoder-Decoder Attention, using Keys and Values from the Encoder and Queries from the previous Decoder layer)⁶.
3. **Feed Forward Network (FFN)**⁷.

B. Sub-Layer Normalization

The output of each sub-layer (Attention and FFN) is followed by an **Add & Norm** layer⁸⁸⁸⁸. This involves a residual connection (Add) and a Normalization layer (Norm). The normalization layers are now typically placed **before** the attention block⁹.

C. Input and Output

The input and output tokens are first converted into **Word Embeddings**¹⁰.

- **Positional Encoding** is added to these embeddings to inject information about the sequence order, as the pure attention mechanism is permutation-invariant (it does not inherently know the order of tokens)¹¹¹¹¹¹¹¹. The Decoder input, or output, is typically shifted right (at inference, the <start> token is input)¹²¹²¹²¹².

2. Positional Encoding: ROPE (Rotary Position Embedding)

While the original paper used sinusoidal positional encodings, these are **no longer used**¹³. Modern approaches like **ROPE** address the need for sequence order information differently.

The goal of ROPE is to **modify the key ($\vec{k}$) and query ($\vec{q}$) vectors** based on their **absolute position (t)** such that the attention score only depends on the **relative position**¹⁴.

The attention score is a dot product, which is related to the cosine of the angle θ between the vectors:

$$\langle \vec{k}, \vec{q} \rangle = \|\vec{k}\|_2 \|\vec{q}\|_2 \cos \theta$$

15

ROPE achieves its position dependence by applying a **Rotation matrix** (R_t) to the $\vec{q}$ and $\vec{k}$ vectors¹⁶¹⁶¹⁶.

For a given channel j at position t , the rotation matrix M_j is defined as:

$$M_j = \begin{bmatrix} \cos(2\pi f_j \cdot t) & -\sin(2\pi f_j \cdot t) \\ \sin(2\pi f_j \cdot t) & \cos(2\pi f_j \cdot t) \end{bmatrix}$$

17

Crucially:

- The frequencies f_j are **fixed** and **not learned**¹⁸.
- Different channels in the vector will have different f_j 's, allowing the model to capture **different timescales** or levels of positional relationship¹⁹.

3. GPT-3 vs. BERT: Architectures and Objectives VS

Modern large language models typically simplify the original Encoder-Decoder design, resulting in two dominant families: **Encoder-only** (like BERT) and **Decoder-only** (like GPT-3) models.

A. GPT-3 (Generative Pre-trained Transformer)

- **Architecture: Decoder-only model**²⁰²⁰²⁰²⁰.
- **Attention:** Uses **Masked/Causal Self-Attention**²¹²¹²¹²¹. This ensures that each position can only attend to its own embedding and those of tokens **earlier in the sequence** (i.e., it only exploits the **left context** of each word)²²²²²²²²²²²²²²²²²²²².
- **Pre-training Objective: Autoregressive generation** (predicting the next token)²³. The

goal at each position is to maximize the probability of the following ground truth token in the sequence using a **Cross-entropy loss**²⁴²⁴²⁴²⁴.

- **Advantage:** Makes **efficient use of the data** since every word contributes a term to the loss function²⁵²⁵.
- **Inference/Use:** Its focus is to **generate the next token in a sequence**²⁶. It uses auto-regressive generation, where the generated token is fed back as input²⁷. Sampling strategies like **Beam search** or **Top-k sampling** are used²⁸²⁸²⁸²⁸.
- **Scale Example:** GPT-3 has **96 transformer layers, 175 billion parameters**, and was trained on **300 billion tokens**²⁹²⁹²⁹²⁹.

B. BERT (Bidirectional Encoder Representations from Transformers)

- **Architecture: Encoder-only model**³⁰.
 - **Attention:** Uses full self-attention, where every token attends to **every other token** in the layer (orange connections)³¹. This allows it to use **both the left and right context** to predict missing words³².
 - **Pre-training Objective (Self-Supervision):**
 1. **Masked Token Prediction:** A small fraction of input tokens are randomly replaced with a [MASK] token, and the model's goal is to predict the missing word using **Cross-entropy loss**³³³³³³³³³³³³³³³³³³³³.
 2. **Sentence Adjacency Prediction:** The model predicts if Sentence B immediately follows Sentence A³⁴³⁴³⁴³⁴. This prediction is typically made on the <sep> token embedding³⁵.
 - **Disadvantage:** This task does **not make efficient use of data**; for example, processing seven tokens might only add two terms to the loss function³⁶.
 - **Inference/Use:** Its goal is to **learn general information about the statistics of language**³⁷. After pre-training, it is adapted to solve a specific task by **fine-tuning**, often by appending a linear layer or MLP to the encoder output (e.g., for sentiment classification using the <cls> token, or named entity recognition using individual word embeddings)³⁸³⁸³⁸³⁸³⁸³⁸³⁸³⁸³⁸³⁸.
-

Test Your Deep Understanding

To ensure your understanding meets the CS182 standard, let's look at the implications of these architectural and objective choices.

Please answer the following three questions based **only on the information presented above and in your notes**:

Question 1: ROPE and Relative Position

The attention score s_{ij} depends on the dot product of the modified query ($\vec{q}_i$) and key ($\vec{k}_j$) vectors.

In ROPE, why must the fixed, unlearned frequencies f_j be different across different channels of the vector? What distinct characteristic of language modeling does this design choice enable the model to capture? ³⁹

Question 2: Trade-offs in Attention

BERT's pre-training task exploits both the left and right context, which is an advantage for representation learning. However, the lecture notes describe this task as having a **disadvantage** regarding data efficiency. Explain, using the information provided, **why** BERT's training is less efficient than GPT-3's autoregressive training, and what primary use case of BERT justifies this efficiency trade-off. ⁴⁰⁴⁰⁴⁰⁴⁰

Question 3: BERT's Dual Task

BERT is trained using two self-supervised tasks: Masked Token Prediction (MTP) and Sentence Adjacency Prediction (SAP). If MTP is already teaching the model about word context, why is the SAP task necessary? What type of language understanding does it provide that MTP alone cannot? ⁴¹⁴¹⁴¹⁴¹

Comments: Gemini now dives deep into each specific topic. I gave the model a lot of leeway to observe how it would want the learning to happen. The short summaries on Transformer architecture, RoPE, and GPT-3 vs BERT are concise and sharp. They provide a good refresh for someone having gone to lecture and wanting to confirm their knowledge. However, I would not recommend such a short summary for someone wanting to use this tool as their only source of learning. In terms of the questions, they can be discerned fairly easily just from the content given in the summaries, but I think this is okay - it requires the student to show they're following along and the future questions will test the student harder

Prompt: 1. The frequencies must be different across different channels of the vector because the model needs to be able to capture different timescales or levels of positional relationship. This enables the model to capture how words are usually dependent on the surrounding words when generating text

2. BERT's training is less data efficient than GPT-3's autoregressive training because adding more tokens doesn't necessarily add more terms to the loss function when training BERT, whereas in GPT-3, each word contributes to the loss function. The primary use case of BERT, to learn general statistical information about languages, justifies this trade-off because we don't care about specific words in this context

3. The SAP task is necessary in this context because the goal is the predict if a sentence follows another one, which needs to be done on the <sep> token. After masked token prediction, we need another algorithm to do this

Response: That's an excellent, comprehensive effort! You've grasped the core differences and functions. Let's refine a couple of points to ensure that deep CS182-level understanding.

1. ROPE and Relative Position

Your answer is spot-on:

- 6.  **Correct:** The different, fixed frequencies ($\mathbf{f_j}$) across channels are used so that **different timescales** or **levels of positional relationship** can be captured.

To make the second half of your answer stronger, try to connect it back to the core math:

- The point of ROPE is to modify the vectors such that the dot product (the attention score) only depends on the **relative position** ($\mathbf{t-i}$) and not the absolute positions (t and i) themselves. The different timescales (frequencies f_j) are the mechanism that allows the model to capture these relative relationships effectively across various lengths.

2. Trade-offs in Attention

This answer is also largely correct and shows great understanding of the trade-off:

-  **Correct:** You correctly identified that in GPT-3, **every word contributes a term to the loss function**, making it more efficient, whereas BERT's Masked Token Prediction (MTP) only adds terms for the few masked tokens. The notes state that processing seven tokens might only add two terms to the loss function.
-  **Correct:** The justification is also correct: this inefficiency is an acceptable trade-off because BERT's primary purpose is **representation learning**—gaining general statistical information about the language—which requires the **bidirectional context** afforded by its full self-attention.

3. BERT's Dual Task (MTP + SAP) 🧠

You correctly identified the mechanism for the Sentence Adjacency Prediction (SAP) task:

-  **Correct:** SAP is required to **predict if a sentence follows another one**, and this prediction is typically made using the embedding from the **<sep> token**.

Here is the key refinement:

- **The Missing Link:** MTP is excellent for learning **local context** (e.g., word choice, grammar, and syntax *within* a sentence or short span). However, it does not explicitly teach the model about the relationship **between** two sentences. The SAP task forces the model to learn about **discourse coherence** and **long-range text organization**. It provides the model with knowledge that spans sentences, which is essential for understanding paragraphs and multi-sentence context.

That was a great run-through! You've demonstrated a strong grasp of the architecture's design principles and the reasoning behind them.

Let's conclude with a final synthesis question that combines the GPT-3 and BERT concepts:

If you were designing a model for a task that **requires** you to identify the relationship between two separate input sentences (like Question Answering, where you read a passage and then a question), which architecture—**Encoder-only (BERT)** or **Decoder-only (GPT-3)**—would be the better starting point based on their pre-training objectives and attention mechanisms, and why?

Comments: The model does a good job of breaking down my questions - I particularly appreciate that it identified I got most of the answers right but also helped me by noting things I could have added to improve my answers. Furthermore, this final synthesis question is a great next-step: it now requires the student to combine the knowledge they should have developed at this point.

Prompt: Masked/causal self-attention enables us to focus on generation because we use every previous word to predict the next word efficiently whereas full self-attention is better for classification because we can refer to the entire sequence

Response: That is a perfect analysis based on the attention mechanisms! You correctly identified **BERT** 🧠 as the better starting point.

Refining the Justification

While the **bidirectional context** (full self-attention) of BERT is critical for creating a rich representation of a single input sequence (like a sentence pair), the most direct evidence from the lecture notes is BERT's unique **pre-training objective**:

7. The **Sentence Adjacency Prediction (SAP)** task specifically trains the model to understand the relationship between two distinct sentences (Sentence A and Sentence B). This is a foundational skill for tasks like **Question Answering** where you need to relate a question (Sentence A) to a passage (Sentence B).
8. GPT-3, conversely, is trained *only* for **autoregressive generation** (predicting the next token) and is therefore focused purely on the *left context* to produce a continuous flow of text.

The combination of **bidirectional attention** and the **SAP pre-training task** makes the Encoder-only (BERT) architecture the superior choice for deep relationship understanding and classification tasks.

Final Synthesis and Confirmation

We have covered the **Core Architecture** (Encoder/Decoder, Add & Norm, FFN), **Positional Encoding** (ROPE's fixed frequencies capturing different timescales based on relative position), and the **GPT-3 vs. BERT** comparison.

Let's use one last question to synthesize the role of the attention mechanism in these two architectures:

Considering the fundamental difference between **Masked/Causal Self-Attention** (GPT-3) and **Full Self-Attention** (BERT), how does the choice of attention mechanism dictate which model is inherently better suited for **generation (GPT-3)** versus **fine-tuning/classification (BERT)**?

Comments: Again, the model does a good job of deeply analyzing my answer. It points out weakness in my justification, particularly going deeper on the tasks each model is preferential for. The final synthesis question puts it all together well

Prompt: To identify the relationship between two separate input sentences, we would prefer to use BERT as a starting point because of its bidirectional properties. Using BERT allows us to refer to both right and left contexts to determine if the sentences are related whereas GPT-3 only enables us to look earlier in the sequence, not at the relationship between two sentences

Response: That's a **perfectly articulated synthesis** 🧠!

You've successfully connected the architectural choice (the attention mask) to the model's intended function and efficiency:

Feature	GPT-3 (Causal/Masked Attention) 🧠🔒	BERT (Full/Bidirectional Attention) 🔗
Why it works for the task	It enforces a left-to-right dependency, which is the definition of generation (predicting the next word).	It allows the model to see the entire context (left and right) for every token, creating a richer, contextualized representation necessary for fine-tuning/classification.
Efficiency Trade-off	Efficient because every word predicted contributes a term to the loss function.	Less efficient because only masked tokens contribute to the loss, but the trade-off is justified by the richer representation.

You've demonstrated a strong, deep understanding of the Transformer architecture, the necessity of positional encoding like ROPE, and the key architectural and objective differences between GPT-3 and BERT, meeting the standard for CS182.

Do you have any final questions about these topics, or would you like to explore another area of deep learning covered in your lectures?

Comments: Good analysis again - would finish here.